Maynooth University
National University of Ireland Maynooth

MSc. Data Science and Analytics Thesis

truncpois: An R Package for Truncated Poisson Distributions

Author: Arun Sundar Paulraj

Student Number: 25250740

Supervisor: Dr. Keefe Murphy

A thesis submitted in fulfilment of the requirements for the degree of Masters in Data Science and Analytics 2025-2026

to the

Department of Mathematics & Statistics

Maynooth University

7th August 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor Dr. Keefe Murphy for his guidance, support and feedback throughout my development of the `truncpois` package and this thesis. I would also like to thank the academic and professional staff both now and in the past of the Department of Mathematics and Statistics at Maynooth University for providing an outstanding learning environment for me and for making my time as an MSc Data Science and Analytics student especially enjoyable and rewarding. Finally, thanks to family and friends for their support, encouragement and patience throughout this journey. I appreciate their belief in my capabilities in helping me complete this work and always will.

Abstract

While there are a number of applications for count data in which some types of observations are simply not made (e.g., when representing zero-truncated counts for studies of species abundance, insurance claims, or reported cases of disease), almost none of the available R implementations for the truncated Poisson, with the exception of the `LaplacesDemon` and `truncdist` packages, offer the functionality needed to work with these distributions; that is to say, they suffer from issues such as having no way to calculate the moments using analytical means for an arbitrary number of parameters, not fully addressing how to compute the probabilities in the right-tail and/or at the log-scale, and relying on Monte Carlo simulation to estimate moments that can be computed exactly.

This dissertation provides an R package `truncpois` that implements all aspects of the (left-, right- and doubly-) truncated Poisson distribution in numerically stable form. It provides all standard density, distribution, quantile, and random sampling functions. Closed form expressions for the mean and variance are derived using the Poisson recurrence relation, while the median and mode are obtained using complementary exact, deterministic methods. The entire computation is performed on the log-scale to avoid overflow with large parameter values. Furthermore, three alternative random sampling algorithms have been developed for use with different truncations. In addition to these functionalities, it also provides a maximum likelihood estimator to obtain the rate parameter from observed counts as well as a visualisation tool to compare the distributions of the truncated and untruncated cases.

Six experiments test the reliability, accuracy, and computational efficiency of this package. These include: visualizing the probability mass function; showing how the truncated moments converge to their non-truncated counterparts; comparing how the sampling methods perform relative to each other; evaluating the computational stability as the parameters take on extreme values while the truncation window is extremely narrow; simulating data from the zero-truncated Poisson distribution; and recovering estimated parameters via maximum likelihood estimation. This demonstrates that `truncpois` fills a significant void in R's available options in analyzing truncated counts with exact, replicable, and computationally efficient algorithms, where previous approximations were incomplete.

Keywords: closed-form moments, maximum likelihood estimation, numerical stability, R package, truncated Poisson distribution.

Generative AI Usage

This project was developed with the assistance of generative AI as a supplementary tool. It was used only to review and enhance the work done independently in the following areas:

- Reviewing and improving the quality of the package implementation, including identifying and correcting mathematical errors in calculations.
- Improving the clarity, wording, and structure of the package's documentation and related supporting materials.

All design, mathematical derivation, statistical techniques, implementation, testing, evaluation and conclusions are independently developed by the author. Every suggestion made using AI was checked and adjusted as needed before being added to this project.

The application of Gen AI in this project can be considered as GenAI editing (as outlined in the MU Student Guidelines for GenAI). AI was only used to review and enhance current content, such as checking its correctness in terms of implementations and improving its clarity. It was not used for the creation of new content or as a basis for decisions on the design, methodology, statistical approaches, and implementation of the project.

The following prompts were used:

- "Is the numerical tolerance to be used for quantile-based support shrinkage in `rtruncpois()` suitable, or does it have the potential to cause problems at extreme parameter values?"
- "Review the Gumbel noise generation step used in the direct sampling method of `rtruncpois()` - is this implemented correctly?"
- "Can you verify whether this formula and its implementation of the standard error of the MLE is correct?"
- "Before I complete this derivation, can you tell me if there's any math mistakes in this derivation?"
- "What could you do to make this Roxygen documentation block more readable?"

Contents

List of tables

List of figures

1 Introduction

Truncated probability distributions form a significant area of research in statistics as they occur quite often, usually resulting from the restricting of the range of valid observations. The truncated Poisson distribution forms an important model for count data and such distributions are increasingly being used for count data arising in species abundance and force of infection or prevalence surveys for example, where the number of cases is never equal to zero (Nadarajah and Kotz 2006).

The topic of truncated distributions has received considerable attention in theoretical literature; for instance, Nadarajah et al. (2006) offer a thorough theoretical treatment of the subject matter, along with R code for evaluating the corresponding truncated distributions. Most importantly, since these implementations are not specialized to handle the case of discrete distributions, they do not properly account for the non-zero probability mass at the lower truncation point; most of the further issues described below are downstream of this one. Various implementations of the truncated Poisson distribution, such as those in the LaplacesDemon and the generic package truncdist (latter is related to Nadarajah and Kotz (2006)), are also inefficient and unstable with respect to the evaluation of the truncated distribution when the parameters are extremely large. In particular, these implementations do not support evaluation on the log scale and also do not allow specifying the upper tail and log-probability arguments, which would allow for an easy evaluation without any instability prone manual modifications. The mean and the variance of the distribution are obtained through numerical integration or Monte Carlo simulation, even though closed form formulas exist for the Poisson distribution.

The `truncpois` R package addresses these gaps in several ways.

One key contribution is that the package provides the full d/p/q/r convention used in base `R`: `dtruncpois()` for the PMF, `ptruncpois()` for the CDF, `qtruncpois()` for the quantile function, and `rtruncpois()` for random sampling.

Another key feature is support for all combinations of the logical arguments `log`, `log.p`, and `lower.tail`. To make this reliable even in extreme cases, the package performs the relevant computations on the log scale and correctly handles inclusive truncation bounds, which is something the code in Nadarajah and Kotz (2006) does not do for discrete distributions.

A further contribution is that `truncpois` computes the mean and variance exactly using closed-form formulas derived from the Poisson recurrence relation. The existing code in Nadarajah and Kotz (2006), by contrast, relies on numerical integration for those quantities, which is not a good fit for discrete distributions; while such quantities could instead be obtained via exhaustive summation over the discrete support, `truncpois` exploits the Poisson recurrence relation instead, avoiding the need for such summation entirely.

The package also includes new exact functions for the median and mode of the truncated Poisson distribution, both computed deterministically rather than by simulation. The package includes a plotting utility as well, so users can compare the truncated distribution directly against the corresponding non-truncated

Poisson case.

Finally, `rtruncpois()` offers three different sampling algorithms, only one of which is available in the earlier work of Nadarajah and Kotz (2006). Section 4 shows how these methods behave in practice and why the extra options are useful.

The special cases of left-truncation, right-truncation, double-truncation and no truncation are all supported across all functions in the package, with suitable defaults used where relevant to correspond to the standard, non-truncated Poisson distribution.

The complete source code for `truncpois` is available on GitHub at https://github.com/arunsundar022/truncpois

The rest of this thesis will be structured as follows. Section 2 will discuss relevant theoretical background concerning the Poisson distribution and discrete univariate distributions, and offer an overview of the current software implementations. Section 3 will discuss the software implementation `truncpois` and the identities upon which the functions in `truncpois` rely. Section 4 will illustrate through experiments the benefits of `truncpois` in comparison to other software implementations. The discussion section will follow in Section 5.

2 Background and Literature Review

This section reviews some of the theory necessary to understand truncated discrete distributions, and the truncated Poisson distribution in particular, before surveying existing software implementations and noting specific shortcomings that `truncpois` is designed to address. Specifically, Section 2.1 introduces truncated distributions, the truncation regimes considered throughout this thesis, and the probability mass function for the truncated Poisson distribution, while Section 2.2 reviews existing R packages that provide some support for truncated distributions, and discusses their limitations relative to `truncpois`.

2.1 Truncated Distributions

When a random variable X generates data that cannot take on any value except for those included in a limited set of possible values (as some of these may be structurally impossible, or merely omitted), this results in a truncated distribution. A truncated distribution is different from censored data (that includes some information about out-of-range values, but not actual values for them); in the case of censoring there is an indication that a value did exist, whereas with truncation, no data is collected that would indicate that such a value exists.

Truncation does not only occur because of the impossibility of observation for some values, but also because of modelling due to the research question posed. For example, when a researcher studies how many modules university students taking part in exchange programs take overseas, there will be truncation from the left as a result of the sampling process since one needs to have at least one module in order to

belong to the sample. In another case, when one surveys tourists visiting a particular place, the researcher might only include those who visited it at least once.

Three truncation regimes are commonly distinguished for a discrete random variable with support constrained to $[a, b]$:

- **Left-truncation** ($a > 0$, $b = \infty$): the most common case in count-data applications, when counts below some threshold cannot be observed. The special case $a = 1$ is known as *zero-truncation*, and comes up whenever the very fact of inclusion in a sample requires at least one occurrence of the event being counted; for example, a respondent will only be included in a survey of hospital visits if they visited hospital at least once.
- **Right-truncation** ($a = 0$, $b < \infty$): counts above some upper threshold are excluded, for instance where a data-collection instrument imposes a maximum recordable value, or where a regulatory limit caps the observable count directly, such as the number of penalty points on a driving licence before it is revoked.
- **Doubly-truncated** ($0 < a \leq b < \infty$): both bounds are finite, to restrict observations to a bounded window, such as a loyalty scheme that only tracks customers who purchased at least one limited-edition item, up to a maximum of ten per customer.

For a truncated Poisson distribution with support $[a, b]$, the probability mass function is

$$P(X = k \mid a \leq X \leq b) = \frac{P(X = k \mid \lambda)}{P(a \leq X \leq b)}, \qquad a \leq k \leq b,$$

with $P(X = k \mid a \leq X \leq b) = 0$ for $k \notin [a, b]$, where $P(X = k \mid \lambda)$ denotes the probability mass function of the corresponding non-truncated Poisson distribution. This is parameterised by a strictly positive rate parameter λ, and its probability mass function is given by

$$P(X = k \mid \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}, \qquad k = 0, 1, 2, \ldots,$$

such that the non-truncated distribution corresponds to the special case $a = 0$, $b = \infty$. The denominator $P(a \leq X \leq b)$ is the normalising constant, and can equivalently be written in terms of the non-truncated cumulative distribution function as $P(a \leq X \leq b) = P(X \leq b) - P(X \leq a - 1)$.

Zero-truncation is sometimes confused with *zero-inflation*, but confusing the two makes little sense in the context of applied statistical modelling. Zero-truncation means that zero has been removed entirely from the sample space (it's impossible for it to be observed), whereas a zero-inflated distribution instead simply adds an excess point mass at zero (over and above that which a standard Poisson distribution would predict) to account for data with more zeroes than a standard model would allow. The `truncpois` package deals only with the former; zero-inflated count models are a different kind of modelling problem, typically fitted with a (different) set of dedicated packages, and will not be addressed here.

Data of this type of count variable that is affected by truncation are fairly common in many different kinds of applications. For example, abundance surveys of species in ecology often do not include all species present in the area being surveyed - those with a zero count have no record of their presence; likewise, insurance claim data will not typically contain any record of claims that did not occur (since zero-claim policies will never generate an actual claim); finally, case counts for diseases are only reported up to a certain threshold number of cases (once reached), so anything below that isn't observed. In all of these cases, fitting an unmodified, naive Poisson model to the data, without taking into account the truncated region of the support of the Poisson distribution, would result in biased parameter estimates (Cameron and Trivedi 2001).

2.2 Existing Software Implementations

Several R packages offer some support for truncated distributions, but none combines Poisson-specific closed-form moments with fully general double truncation and complete log-scale support. Five are considered here.

LaplacesDemon. The `LaplacesDemon` (Statisticat, LLC 2026) package implements the truncated Poisson distribution through the functions `dtrunc()`, `ptrunc()`, `qtrunc()`, and `rtrunc()` (with `spec = "pois"`), respectively, for the density, distribution, quantile, and random generation. Values of the PMF and CDF of the truncated distribution match those of `truncpois` only when $a = 0$, since `LaplacesDemon` does not make the $a \to a - 1$ adjustment needed for left- or doubly-truncated cases, discussed further below; there are also a number of further disadvantages that influenced the choice of implementation of truncpois. Namely, the `ptrunc()` and `qtrunc()` functions assume a number of properties such as `lower.tail = TRUE`, `log = FALSE`, and `log.p = FALSE`, among others, and do not properly handle the `lower.tail` and `log.p` arguments when they are set to values other than these defaults. As a result, in order to calculate the upper-tail probability or the probability in the log-scale, one needs to perform manual calculations such as $(1 - \text{p})$ or $\log(1 - \text{p})$. However, this approach is inaccurate at best, and possibly dangerous. It is not just imprecise, but plain wrong. Functions of truncated distributions implemented in `LaplacesDemon` are defined in terms of a series of calls to the corresponding non-truncated Poisson functions (g and G from Section 3, respectively), and the final modification by `1 - p` or `log(1 - p)` does not work properly with such chain of calculations and might lead to the completely wrong results rather than just inaccurate ones. There are also no analytical expressions provided for the mean and variance of the truncated Poisson distribution; obtaining these instead requires either simulation via the `rtrunc()` function, which introduces additional sampling error depending on the extremeness of the truncation parameters, or numerical integration over the discrete probability mass function, neither of which is as appropriate as the closed-form, recurrence-based approach used by `truncpois` (Section 3). Furthermore, there are no functions provided in `LaplacesDemon` for calculating the median, as well as, beyond, the mode.

truncdist. Nadarajah and Kotz (2006) also published an accompanying R package, `truncdist` (Novomestky and Nadarajah 2016), that offers generalized (distribution-agnostic) `dtrunc()`, `ptrunc()`,

`qtrunc()`, `rtrunc()` wrapper functions for truncating any distribution supplied by `stats`, `stats4`, or `evd`. While this level of generality is useful for certain purposes, it makes `truncdist` unsuitable for this work, since there is no way to leverage the particular closed-form recurrence relation of the Poisson distribution that `truncpois` relies on for exact moments, nor does `truncdist` provide a path for computations on the log scale. As a package built on the same general principles as `LaplacesDemon`, `truncdist` inherits the same limitations listed above: it assumes `lower.tail = TRUE`, `log = FALSE`, and `log.p = FALSE` throughout, and provides no closed-form median or mode.

VGAM and VGAMdata. The `pospoisson()` family in the `VGAM` package (Yee 2025a) allows fitting of a *zero-truncated (or positive) Poisson* regression using `vglm()`. Its companion `VGAMdata` package (Yee 2025b) provides the corresponding standalone distribution functions, `dpospois()`, `ppospois()`, `qpospois()`, and `rpospois()`, but this is restricted to the zero-truncated case; it does not support right-truncation or double-truncation, and inherits the same `lower.tail`, `log.p`, and `log` restrictions from above.

gamlss.tr and countreg. The two remaining packages use truncation to create new distributions for use in regression modelling, rather than for use as stand-alone utilities of a specific truncated distribution. `gamlss.tr` (Stasinopoulos and Rigby 2024) uses `gen.trun()` to create a truncated version of any `gamlss.family` distribution for use in a GAMLSS regression model. `countreg` (Zeileis and Kleiber 2024) (currently available on R-Forge) has the option of fitting zero-truncated count regression models via either `zerotrunc()` or `ztpoisson`, each of which can be used as the count component of a hurdle model. Neither package provides the exact closed form moments of a stand-alone (non-regression) truncated Poisson distribution, which forms the focus of this thesis, and both inherit the log-scale and tail probability limitations discussed above.

Another limitation common to all five of the packages just discussed (in addition to the `log`, `log.p`, and `lower.tail` issues and their reliance on numerical integration or Monte Carlo simulation for their moments) is that none of them make a correction to the lower truncation bound for the discrete nature of the distribution. For a continuous distribution, $P(X = a) = 0$ so there's no need for any such adjustment; however for a discrete distribution like the Poisson, $P(X = a) \geq 0$ and so if one wants to compute $P(a \leq X \leq b)$ correctly one needs to evaluate the non-truncated CDF at $a - 1$, not at a, so as to include (rather than exclude) the probability mass located exactly at the lower truncation bound. `truncpois` makes this $a \to a - 1$ adjustment throughout; none of the packages reviewed above do so, which introduces yet another unacknowledged source of error whenever they are applied to the left-truncated or doubly-truncated cases.

As LaplacesDemon is the most popular package of all packages analyzed in this thesis and the one that is most aligned with the original algorithm of Nadarajah and Kotz (2006), further comparison in the rest of this thesis will focus mainly on LaplacesDemon.

3 Package Implementation

This Section describes each of the ten functions exported by `truncpois`, focusing on the mathematical identities on which their implementation is based and the key differences relative to the code that accompanied Nadarajah and Kotz (2006). $f_X(x; a, b, \lambda)$, $F_X(x; a, b, \lambda)$, and $F_X^{-1}(p; a, b, \lambda)$ denote the probability mass function, cumulative distribution function, and quantile function, respectively, of the truncated Poisson distribution targeted throughout this thesis, and $g(x; \lambda)$, $G(x; \lambda)$, and $G^{-1}(p; \lambda)$ denote the corresponding functions of the non-truncated Poisson distribution, implemented in base `R` as `stats::dpois()`, `stats::ppois()`, and `stats::qpois()` respectively. Further implementation detail for every function, beyond what is summarised here, is available in the associated `roxygen2` (Wickham et al. 2026) help documentation distributed with the `truncpois` package itself.

3.1 `dtruncpois`

The probability mass function of the truncated Poisson distribution is

$$f_X(x; a, b, \lambda) = \frac{g(x; \lambda)}{G(b; \lambda) - G(a-1; \lambda)}, \qquad a \leq x \leq b,$$

implemented by `dtruncpois()` entirely on the log scale, $\log f_X(x) = \log g(x; \lambda) - \log\big[G(b; \lambda) - G(a-1; \lambda)\big]$, with the log-scale difference in the denominator computed via a numerically stable log-difference-of-exponentials routine rather than direct subtraction; that is, the subtraction itself is carried out on the log scale, rather than by exponentiating $G(b; \lambda)$ and $G(a-1; \lambda)$ back to the linear scale first. Two differences relative to Nadarajah and Kotz (2006) stand out. The normalising constant is computed on the log scale, as just discussed, rather than as a direct (and potentially unstable) difference of the untruncated CDF evaluated at the bounds. `dtruncpois()` also allows `x` to take values outside $[a, b]$, returning a density of 0 (or $-\infty$ if `log = TRUE`), rather than raising an error, which is convenient if `x` is a vector containing values both inside and outside the truncation region, and is more user-friendly than `LaplacesDemon`, which raises an error in this situation.

3.2 `ptruncpois`

The cumulative distribution function is defined as

$$F_X(x; a, b, \lambda) = \frac{G(x; \lambda) - G(a-1; \lambda)}{G(b; \lambda) - G(a-1; \lambda)}, \qquad a \leq x \leq b,$$

with $F_X(x) = 0$ for $x < a$ and $F_X(x) = 1$ for $x \geq b$. Compared to the function `ptruncpois()` in Nadarajah and Kotz (2006), the function `ptruncpois()` differs in four aspects: (a) it handles cases where the argument `log.p = TRUE` and/or `lower.tail = FALSE`, in addition to the default of lower-tail and linear-scale probability; (b) it computes the normalising constant in the logarithmic scale; (c) the input value of q can take values outside $[a, b]$ without errors and in such cases, the corresponding

truncated probability of 0 or 1 (or $-\infty$ or 0 on the log scale) is returned, again more user-friendly than `LaplacesDemon`, which raises an error in this situation, and (d) the value of $a-1$ is used in evaluating $G(\cdot)$, for the same reason mentioned in Section 2.2. Point (a) is particularly important to note here: when the CDF is defined as above in terms of $G(\cdot)$, all the internal evaluations of $G(\cdot)$ in computing the probability $F_X(x)$ are done in logarithmic scale with lower tail only, and then the `lower.tail` and `log.p` options are applied exactly once to the aggregated result, as opposed to being applied individually to each constituent evaluation of $G(\cdot)$. On the other hand, the function in Nadarajah and Kotz (2006) applies the requested `lower.tail` and `log.p` options directly to each constituent call of the underlying non-truncated distribution function $G(\cdot)$, since $F_X(x)$ involves evaluating the ratio of two such functions, this method does not propagate the `lower.tail` and `log.p` options properly during intermediate calculations, leading to results that may not be just inaccurate but incorrect.

3.3 `qtruncpois`

Inverting the relationship above for a given lower-tail probability p, the value x such that $F_X(x) = p$ can be given directly by

$$G(x;\lambda) = G(a-1;\lambda)\,(1-p) + p\,G(b;\lambda),$$

so that

$$F_X^{-1}(p;a,b,\lambda) = G^{-1}\big(G(a-1;\lambda)(1-p) + p\,G(b;\lambda);\ \lambda\big),$$

where $G^{-1}(\cdot)$ is the untruncated quantile function. This identity enables the function `qtruncpois()` to call `stats::qpois()` directly after remapping the target probability, rather than requiring a bespoke search. As with `ptruncpois()`, the differences relative to Nadarajah and Kotz (2006) are: (a) support for `log.p = TRUE` and/or `lower.tail = FALSE`; (b) the log-scale computation of the remapped probability above, using a log-sum-exp routine rather than direct addition, and (c) the $a \to a-1$ coercion. In respect of (a), the function `qtruncpois()` first converts whichever combination of `lower.tail` and `log.p` is supplied onto a single log-scale lower-tail probability, and then applies the remapping formula above; the input `p` is therefore assumed, by the time it gets to $G(\cdot)$ and $G^{-1}(\cdot)$, to already represent the correct tail and scale. This is in contrast to the approach of Nadarajah and Kotz (2006) which supplies the `lower.tail`/`log.p` arguments to each individual $G(\cdot)$ and $G^{-1}(\cdot)$ call, rather than taking account of them once in the overall expression.

3.4 `rtruncpois`

Three sampling algorithms are provided via the `method` argument, only one of which is provided by Nadarajah and Kotz (2006).

The **direct** method sets out the finite support and samples via the Gumbel-max approach (Yellott 1977): for a categorical distribution with log-probabilities $\{\log f_X(k)\}_{k=a}^{b}$, adding independent standard Gumbel noise γ_k to each log-probability and taking the arg max, $\arg\max_k\big[\log f_X(k) + \gamma_k\big]$, produces a draw from

the categorical distribution itself, without ever having to compute the normalising constant explicitly. `truncpois` generates the required Gumbel noise from draws from an exponential distribution, by virtue of the identity

$$-\log(\text{Exp}(1)) \sim \text{Gumbel}(0, 1).$$

Because this method enumerates $[a, b]$ explicitly, it is, in principle, unusable when $b = \infty$; `rtruncpois()` resolves this by first shrinking the effective support to $[\max(a, q_{\text{lo}}), \min(b, q_{\text{hi}})]$, where q_{lo} and q_{hi} are extreme quantiles of the untruncated distribution, corresponding to a tail probability of $\sqrt{\varepsilon}$, where ε is machine epsilon (normally `2.220446e-16` on standard machines), obtained via $G^{-1}(\cdot)$. This explains why the "direct" method is usable even when $b = \infty$: the probability mass in the ignored tail is so small that it is not representable in floating-point arithmetic; despite the cost of the expanded enumeration that this incurs, "direct" is nonetheless the default method in the package, and Section 4 then compares its computational efficiency against the other two methods across a range of scenarios.

The **inversion** method applies the quantile-inversion identity of `qtruncpois()`, mentioned above, to a uniform random draw: for $U \sim \text{Uniform}(0, 1)$, $F_X^{-1}(U; a, b, \lambda)$ is a draw from the truncated distribution. Rather than generate U and compute $\log(U)$ directly, `truncpois` instead draws the equivalent log-scale value via the identity $\log(\text{Uniform}(0, 1)) \sim -\text{Exp}(1)$ so that a single exponential draw can be used to supply the log-probability input it needs internally, avoiding the loss of precision that can come from computing $\log(\texttt{runif}(n))$ directly when U is close to 0, or needing to exponentiate prematurely. This method never enumerates the support so it works for any a, b, and λ, including $b = \infty$. This is the same method employed by `LaplacesDemon`, except that `LaplacesDemon` uses plain (linear-scale) inversion rather than the $-\text{Exp}(1)$ identity described above.

The **bounded** method also inverts through the CDF, but avoids `qtruncpois()`'s remapping step entirely: it instead draws a value on the log scale directly within $[\log G(a-1; \lambda), \log G(b; \lambda)]$ (using the same $\log(\text{Uniform}(0, 1)) \sim -\text{Exp}(1)$ identity where $G(a-1; \lambda) = 0$, but a numerically-stable $\log(x+y)$ construction otherwise), and then passes this directly to the *untruncated* quantile function $G^{-1}(\cdot)$, following the general rejection-free approach to bounded-interval inversion that Zhang and Reiter (2022) describe. Because the resulting value on the log scale is already within the correct sub-interval, no truncated normalising constant, or remapping via `qtruncpois()`, is needed on each draw. This is why `"bounded"` is the fastest of the three methods whenever the truncation region retains most of the untruncated probability mass, as demonstrated empirically in Section 4.

3.5 `extruncpois` and `vartruncpois`

The mean and variance are grouped together in this subsection, since both are obtained from the same underlying Poisson recurrence relation, $k\, g(k; \lambda) = \lambda\, g(k-1; \lambda)$. Summing both sides over $k \in [a, b]$ gives

$$\sum_{k=a}^{b} k\, g(k; \lambda) = \lambda \sum_{k=a}^{b} g(k-1; \lambda) = \lambda[G(b-1; \lambda) - G(a-2; \lambda)],$$

so that, after normalising by $P(a \le X \le b) = G(b;\lambda) - G(a-1;\lambda)$,

$$E[X \mid a \le X \le b] = \lambda \, \frac{G(b-1;\lambda) - G(a-2;\lambda)}{G(b;\lambda) - G(a-1;\lambda)}.$$

An analogous argument using the second-order recurrence $k(k-1)\, g(k;\lambda) = \lambda^2\, g(k-2;\lambda)$ gives

$$E[X(X-1) \mid a \le X \le b] = \lambda^2 \, \frac{G(b-2;\lambda) - G(a-3;\lambda)}{G(b;\lambda) - G(a-1;\lambda)},$$

from which $E[X^2] = E[X(X-1)] + E[X]$ and $\mathrm{Var}(X) = E[X^2] - E[X]^2$ follow directly. Both `extruncpois()` and `vartruncpois()` evaluate these expressions via log-scale differences of `stats::ppois()` calls, so no explicit summation over the support is ever performed and the closed forms above are exact regardless of how wide $[a, b]$ is. This represents a substantial departure from Nadarajah and Kotz (2006), whose code instead relies on numerical integration to obtain the mean and variance; numerical integration is a technique designed for continuous functions, and is not, in general, an appropriate way to sum a discrete probability mass function. Exact summation over the support would be a valid discrete alternative, but can only be done when the support is finite (i.e. whenever b is finite, regardless of a); the recurrence-based approach used by `truncpois` avoids this restriction entirely, remaining exact and equally cheap regardless of whether b is finite or infinite. `stats::ppois()` also returns 0 whenever its argument is negative, so the $a-1$, $a-2$, and $a-3$ terms that appear throughout the formulas above are automatically and safely handled without having to special-case them in the implementation, even when they fall below the support.

3.6 `medtruncpois` and `modtruncpois`

Median and modes are discussed in their own section as these are completely new contributions to what was provided by Nadarajah and Kotz (2006), whose code did not compute either of these quantities. Median can be calculated by $F_X^{-1}(0.5; a, b, \lambda)$ using the same inverse formula of the quantile function that was used for `qtruncpois()`. The `medtruncpois()` calculates the above-mentioned inverse quantile function based on the truncated CDF evaluated at the two truncation points while staying in the log scale all the way to call `qtruncpois()` to get the quantile.

The modes of the non-truncated Poisson distribution have been widely recognized as $\lfloor \lambda \rfloor$ (with the tied modes at $\lambda - 1$ if λ is a positive integer as the ratio $g(k;\lambda)/g(k-1;\lambda) = \lambda/k$ is equal to 1 when $k = \lambda$ in this situation) and `modtruncpois()` calculates the modes of the non-truncated Poisson distribution using the above formula and then clips the results to $[a, b]$ (hence returns every value $k \in [a, b]$ that is a mode after clamping with a warning issued in case there are more than one of those values).

3.7 `plottruncpois`

`plottruncpois()` plots the PMF, CDF, or quantile function of the truncated Poisson distribution, chosen by the user using the `type` parameter, with an option to also plot the non-truncated Poisson distribu-

tion (`compare = TRUE`, which is the default) for comparison purposes. Under the hood, it always uses `dtruncpois()`, `ptruncpois()`, and `qtruncpois()` functions with their `log`, `log.p`, and `lower.tail` parameters set to their default values of `FALSE`, `FALSE`, and `TRUE`, respectively (these are not yet available as parameters for `plottruncpois()`, but are planned for a future version of the package), and in case of CDF plotting, sets the lower limit of the horizontal axis to 0 rather than to a to visualize the impact of the left truncation. The plot type for the PMF is shown in Section 4, while the other two plot types are shown below.

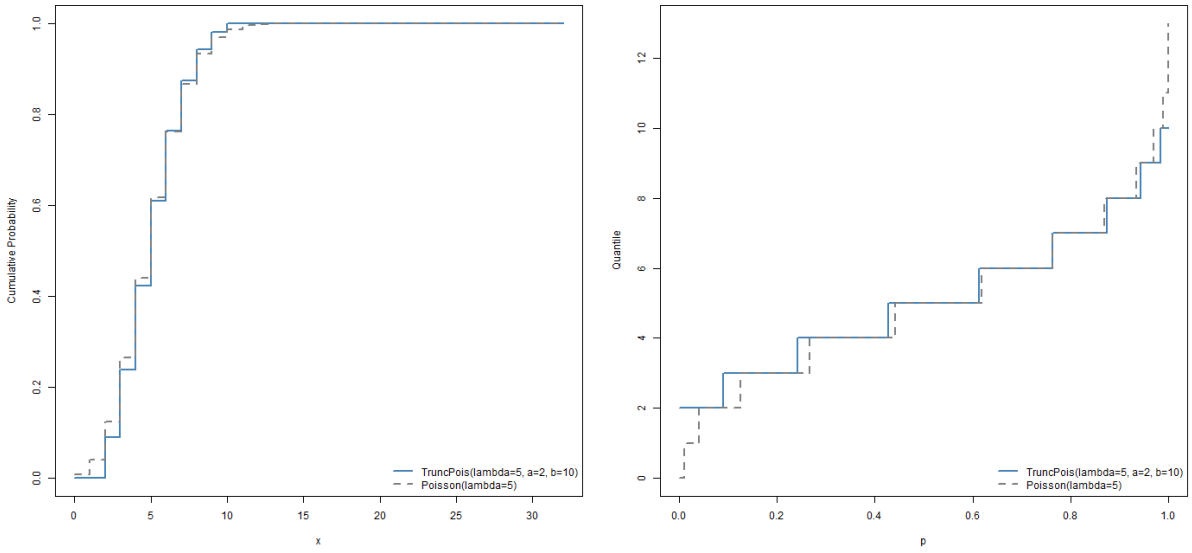

Figure 1: CDF and quantile function plots via `plottruncpois()`, comparing TruncPois($\lambda = 5$, $a = 2$, $b = 10$) against Poisson($\lambda = 5$).

No equivalent visualisation utility is provided by Nadarajah and Kotz (2006) or by any of the packages reviewed in Section 2.2.

3.8 `mletruncpois`

Given a sample $x_1, \ldots, x_n$ drawn independently from a truncated Poisson distribution with known bounds $[a, b]$, the log-likelihood is

$$\ell(\lambda) = \sum_{i=1}^{n} \log f_X(x_i; a, b, \lambda),$$

which, in code, is `sum(dtruncpois(x, lambda, a, b, log = TRUE))`. `mletruncpois()` maximises the log-likelihood $\ell(\lambda)$ over λ by minimising the negative log-likelihood numerically, using one-dimensional Brent optimisation (Brent 1973) via `stats::optimize()`, seeded from a search interval around the mean of the sample. This is practical here, because the non-truncated Poisson log-likelihood is of course well known to be strictly concave in λ and truncation only rescales the likelihood by a λ-dependent normalising constant, meaning that it does not introduce any additional multi-modality that might defeat a simple one-dimensional search. `mletruncpois()` also returns the standard error of $\hat{\lambda}$, which can be obtained from the curvature (second derivative) of the negative log-likelihood at the optimum: writing H for

the (scalar) Hessian of the negative log-likelihood at $\hat{\lambda}$, evaluated numerically via the `numDeriv` package (Gilbert and Varadhan 2019), the standard error is $\text{se}(\hat{\lambda}) = \sqrt{1/H}$, the usual asymptotic result for a maximum-likelihood estimator.

Each function described above checks the validity of its inputs before proceeding: `lambda` must be positive, the truncation bounds must all satisfy $a \geq 0$, $b > 0$, $a < b$, and both must be integers (or $b = \infty$), and `rtruncpois()` requires scalar (rather than vector) parameters, since it is not vectorised over λ, a, or b the way `dtruncpois()`, `ptruncpois()`, and `qtruncpois()` are. Invalid inputs cause an informative `stop()` with a descriptive error message; recycling of vector-valued `lambda`, `a`, or `b` arguments to a common length in `dtruncpois()`, `ptruncpois()`, and `qtruncpois()` triggers a `warning()` rather than silent recycling, to guard against accidental misuse.

3.9 Software Engineering

The package follows the standard `R` (R Core Team 2026) conventions for packages to make it easy to maintain and install. Source code is organised by functionality: `truncpois.R` implements the four core distribution functions (`dtruncpois()`, `ptruncpois()`, `qtruncpois()`, `rtruncpois()`), `moments.R` implements the closed form mean, variance, median, mode, `mle.R` implements `mletruncpois()`, and `plottruncpois.R` implements the visualisation utility. A further unexported file, `zzz_utils.R`, holds a small set of private helper functions (private in the sense of being unexported and undocumented) that are central, implemented and tested once, rather than duplicated, across functions: `.log_sum()` and `.log1mexp()` compute the log-sum-exp and log-$(1 - \exp(\cdot))$ identities respectively (see also the `Rmpfr` package (Maechler 2025), which provides similar `log1mexp()` and `log1pexp()` functions), `.log_diff()` computes a numerically stable log-difference-of-exponentials used throughout for subtracting two log-scale probabilities, `.log_denom_truncpois()` computes the log-scale normalising constant $G(b;\lambda) - G(a-1;\lambda)$ shared by the core distribution functions, and `.gumbel_max()` implements the Gumbel-max sampling step used by `rtruncpois()`'s `"direct"` method.

All exported functions are thoroughly documented via `roxygen2` (Wickham et al. 2026) with executable `@examples` code blocks that double as usage examples, and their functionality is validated through an automated `testthat` (Wickham 2011) test suite comprising checking of correct handling of boundaries (i.e. of PMF summing to exactly 1 within the support, and returning exactly 0 outside of it), agreement of `ptruncpois()` and `qtruncpois()` with the equivalent (with default arguments) functions from the `stats::dpois()`, `ppois()`, `qpois()`, `rpois()` families, being the inverse of `ptruncpois()` and `qtruncpois()`, large-sample consistency of `rtruncpois()` to closed form expressions for the moments for all three sampling algorithms, input validation, and a separate stress-test file that tests various corner cases of the parameters including the cases where $\lambda = 100{,}000$ and minimal possible truncation window (demonstrated in Section 4). The sole hard dependency of the `truncpois` package aside from `base R` is `numDeriv` (Gilbert and Varadhan 2019), which is used by `mletruncpois()` to compute the Hessian of the log-likelihood; `LaplacesDemon`, `microbenchmark` (Mersmann 2024), `knitr` and `rmarkdown` are `Suggests` dependencies only, respectively used in comparisons demonstrated in Section 4, benchmark-

ing and vignette generation. The package's code is further validated with `styler` (Müller et al. 2025) to ensure uniform formatting. `goodpractice` (Padgham et al. 2026) provides additional best practice recommendations, and `desc` (Csárdi et al. 2023) is used to programmatically validate the `DESCRIPTION` file. `devtools` (Wickham et al. 2022) and `usethis` (Wickham et al. 2025) were used throughout to support package development. The `truncpois` package is distributed under the GNU General Public License (version ≥ 3). The project is maintained using version control with `git` and is publicly available on GitHub at https://github.com/arunsundar022/truncpois, which also hosts an informative `README.md` file, itself generated from an intermediary RMarkdown file so as to include representative, executable examples of the code in use.

4 Results and Experiments

A set of six experiments provides evidence of the reliability, precision, and computational speed of `truncpois`. In Experiment 1, the PMF of the truncated Poisson distribution is plotted along with the non-truncated distribution. Experiment 2 illustrates the convergence of the truncated moments towards their non-truncated equivalents in the case of closed-form solutions as the truncation window grows larger; since the `LaplacesDemon` package lacks closed-form solutions for the moments of the truncated Poisson distribution, this experiment offers no direct way to contrast the two. Experiment 3 evaluates the computational efficiency of the three sampling algorithms of the package and `LaplacesDemon` under five different truncation settings. Experiment 4 tests the numerical stability when the parameters take extreme values and the truncation window is small, and contrasts the results with those of `LaplacesDemon`; this experiment uncovers several shortcomings of the latter, which are addressed in Section 2.2. Experiment 5 samples data from a zero-truncated Poisson distribution and validates the sampling and moment functions of the package against it. Experiment 6 estimates a known rate parameter using `mletruncpois()` on simulated data.

4.1 Experiment 1: visualisation of Probability Mass Function

This experiment shows how the PMF of the non-truncated Poisson distribution differs from the PMF of a right-truncated Poisson distribution, generated directly via the package's own `plottruncpois()` visualisation utility; the truncation bounds are shown directly in the figure rather than in a plot title, consistent with the captioning convention used throughout this thesis.

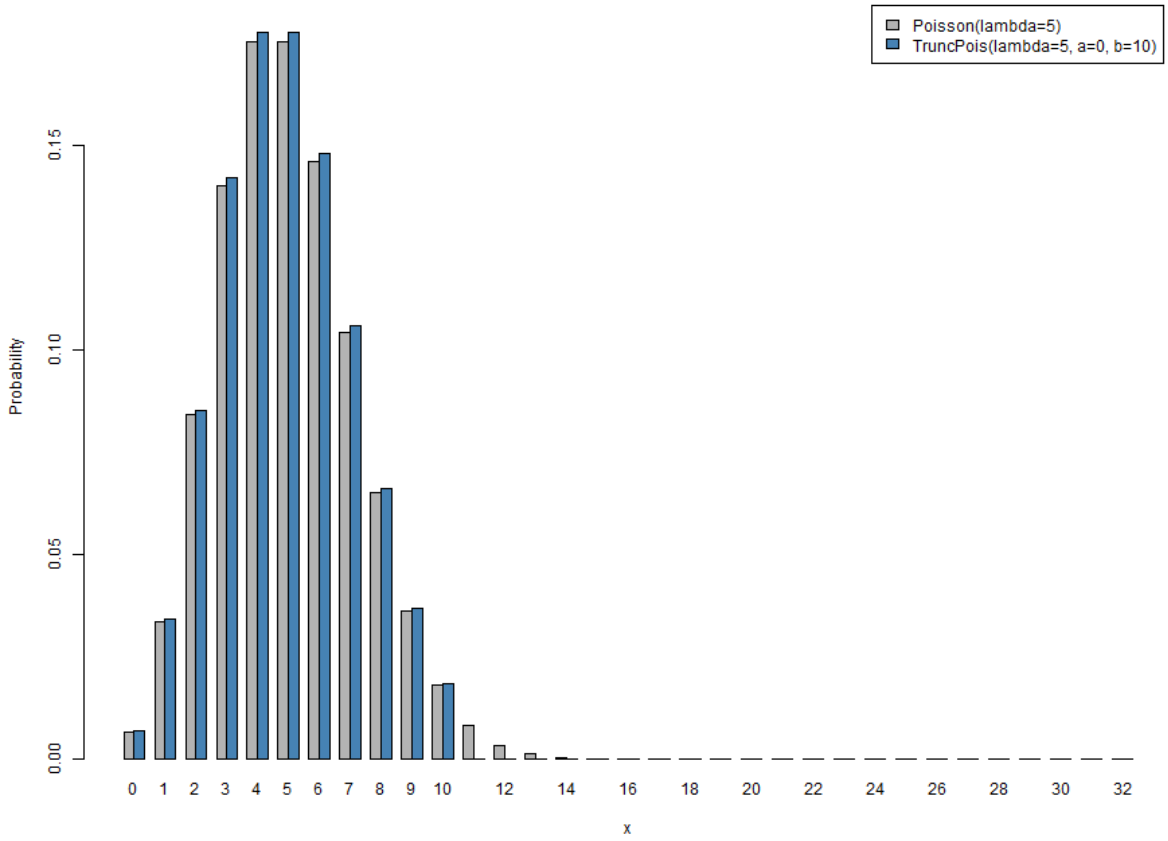

Figure 2: PMF of the non-truncated and right-truncated Poisson distribution ($\lambda = 5$, $b = 10$).

Truncation concentrates the probability mass onto the interval $[0, 10]$. Because right-truncation alone rescales the entire PMF by the same constant factor $1/G(b; \lambda)$, the absolute increase in probability at each point is proportional to the original, non-truncated density; the largest visible increase therefore occurs near the mode ($x = 4$ and $x = 5$), while the increase is smallest near the excluded tail close to $b = 10$.

4.2 Experiment 2: moment convergence

As the truncation boundaries become larger, the moments of the truncated distribution tend towards the corresponding moments of the untruncated distribution. The following experiment demonstrates the convergence of mean and variance.

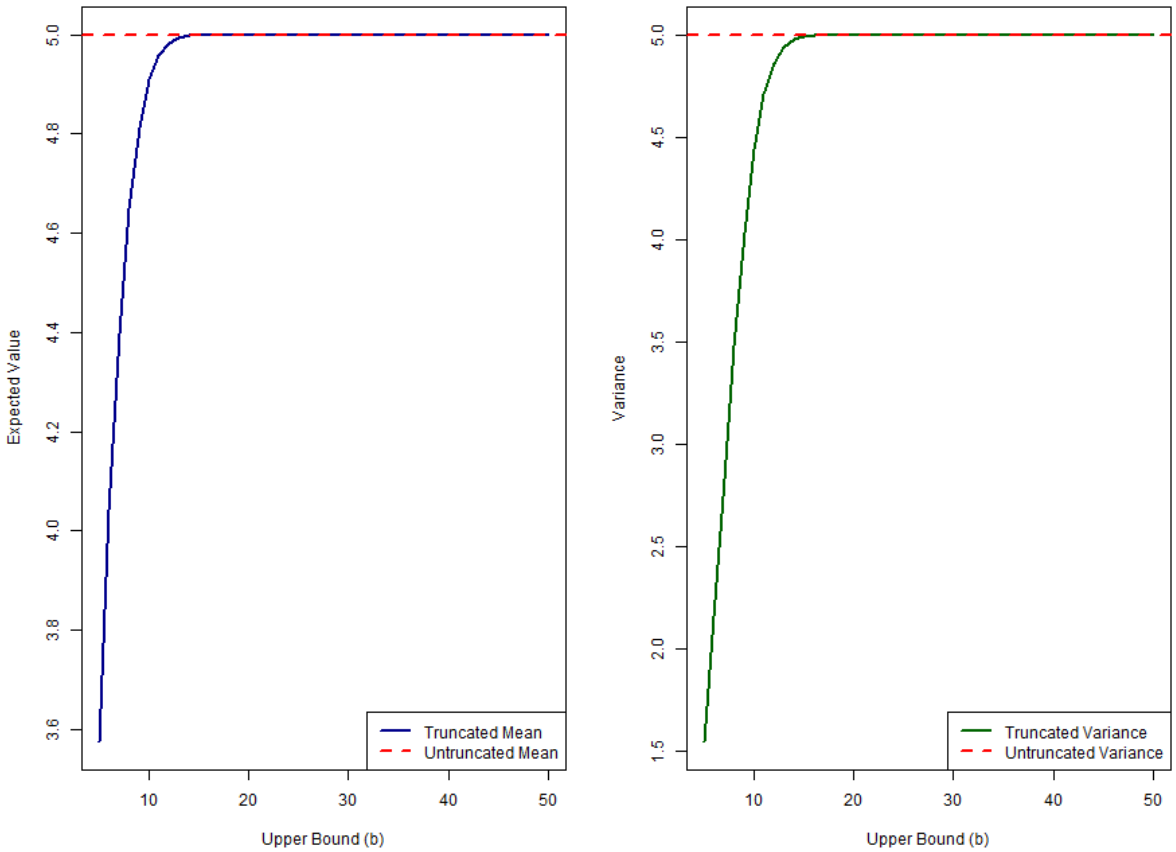

Figure 3: Mean and variance convergence of a right-truncated Poisson($\lambda = 5$) distribution as the upper bound b increases.

Both the mean and the variance of the right-truncated distribution converge to $\lambda = 5$ as the upper bound b grows, recovering the untruncated Poisson moments once b is large enough that truncation removes a negligible share of the probability mass.

Table 1 gives the exact closed-form mean and variance at several selected upper bounds; these were computed directly with `extruncpois()` and `vartruncpois()`. Both moments are clearly below their nontruncated values of 5 for small b since restricting the support to $[0, b]$ removes the right tail of the distribution and pulls down the conditional mean and variance; by $b = 20$ both moments have converged to the nontruncated values to within rounding precision since the nontruncated Poisson(5) puts negligible probability above 20.

More generally, truncation always reduces the variance of the distribution, regardless of which side is truncated, since restricting the support will necessarily remove some probability mass from at least one tail. The effect on the mean, however, depends on the direction of truncation: right-truncation, as observed here, pulls the conditional mean downwards (by removing the upper tail), while left-truncation has the opposite effect of pulling the conditional mean upwards (by removing the lower tail; and in the case of zero truncation, the point mass at $X = 0$).

Table 1: Exact mean and variance of the right-truncated Poisson($\lambda = 5$, $a = 0$, b) at selected upper bounds, computed via `extruncpois()` and `vartruncpois()`.

Upper bound b	5	10	20	30	50
Mean	3.576	4.908	5.000	5.000	5.000
Variance	1.547	4.440	5.000	5.000	5.000

No direct comparison to `LaplacesDemon` is shown for this experiment, since it provides no closed-form mean or variance for the truncated Poisson distribution at all; the only way to approximate these moments via `LaplacesDemon` is by simulating a large sample from `rtrunc()` and computing the sample mean and variance, which is strictly less precise, and slower, than the exact values reported here.

4.3 Experiment 3: sampling method efficiency

The `rtruncpois()` function provides three sampling algorithms optimized for different parameter configurations. This experiment compares their computational efficiency, and that of `LaplacesDemon::rtrunc()`, across five diverse truncation scenarios using `microbenchmark::microbenchmark()`: a left-truncated (zero-truncated) case with unbounded support, a wide right-truncated case, a narrow doubly-truncated case with λ centered in the window, a narrower doubly-truncated case with λ near the edge of the window, and a doubly-truncated case with λ positioned well outside the truncation window entirely.

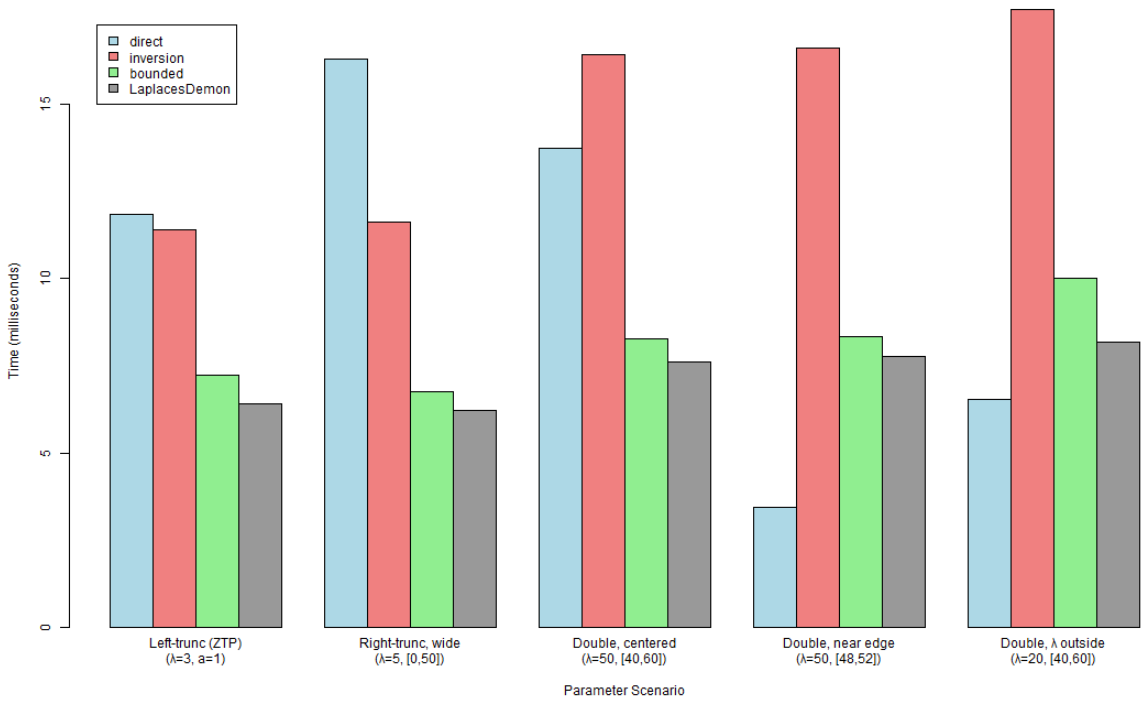

Figure 4: Sampling method efficiency benchmark across five truncation scenarios, comparing `truncpois`'s three methods against `LaplacesDemon::rtrunc()`.

Table 2 reports the median wall-clock time (milliseconds) to draw 10,000 samples under each method and scenario, taken over 20 `microbenchmark` repetitions per cell (i.e. a fresh set of 10,000 samples is drawn in each of the 20 repetitions, and the median of those 20 timings is reported). `LaplacesDemon::rtrunc()` was called with a manually corrected to $a - 1$ throughout, so that the timing comparison is made on equal, correct terms.

Table 2: Median time (ms) to draw 10,000 samples per method and scenario, over 20 `microbenchmark::microbenchmark()` repetitions. Scenarios are as described above: ZTP (λ=3, a=1), wide right-truncation (λ=5, [0,50]), centered double-truncation (λ=50, [40,60]), near-edge double-truncation (λ=50, [48,52]), and λ outside the window (λ=20, [40,60]).

Method	ZTP	Right (wide)	Double (centered)	Double (near edge)	λ outside
Direct	11.89	15.35	14.33	3.66	7.29
Inversion	9.97	10.69	14.79	15.69	16.77
Bounded	5.51	5.58	7.07	6.99	8.43
LaplacesDemon	4.71	4.90	6.64	6.32	7.39

`LaplacesDemon`'s `rtrunc()` is the fastest of the four methods compared in three of the five scenarios tested here (ZTP, wide right-truncation, and centered double-truncation), since it does the same style of direct CDF-inversion (at lower overhead from the lack of input validation) as `truncpois`'s `"bounded"` method; in the remaining two scenarios (near-edge and λ-outside double-truncation), `truncpois`'s `"direct"` method is fastest overall instead, for the reasons discussed below. Raw sampling speed was never one of the main things `truncpois` was pitched for, but this comparison comes with two important caveats. `rtrunc()` only produced these timings once a was manually corrected to $a - 1$ in each of the scenarios of this table; if passed naïvely (as a user unaware of the difference shown in Section 2.2 would do) it instead silently returns samples from the wrong distribution (and doesn't raise an error), a failure mode `truncpois` doesn't have. `LaplacesDemon` also doesn't provide any way of getting closed-form moments or medians, modes, or maximum-likelihood estimates from the samples it does generate, all of which `truncpois` provides directly. Of `truncpois`'s own three methods, `"bounded"` is the fastest in three of the five scenarios (ZTP, right-truncated wide, and double-centered), since, like `LaplacesDemon`, it performs a single vectorized CDF-inversion pass through the non-truncated Poisson quantile function, never enumerating the support nor remapping through `qtruncpois()`. The cost of the `"direct"` method is more variable since it has to first enumerate the finite support and generate a Gumbel variate per sample per support point: it is the fastest of all four methods compared, including `LaplacesDemon`, in the remaining two scenarios (*near edge*, where it has only five support points, and λ *outside*), while it is the slowest of all four methods instead in the two scenarios with the widest effective enumerated support (*ZTP* and *right-truncated, wide*). Notwithstanding this variability, `"direct"` is the package's default method: it samples in direct proportion to the values `dtruncpois()` itself returns, making it correct-by-construction whenever the PMF is correct without depending on whether `stats::qpois()`'s numerical search successfully inverts an extreme log-probability, as both `"inversion"` and `"bounded"` do. Of the three `truncpois` methods tested here, `"inversion"` is the slowest in the other three scenarios (double-centered, near-edge, and λ-outside), because it has to incur the overhead of `qtruncpois()`'s probability-remapping step on every

draw, on top of sharing the same reliance on `stats::qpois()` that `"bounded"` has; it is kept chiefly as an example of a simple, straightforward implementation that is based directly on `qtruncpois()`. Overall, it can be concluded that the performance of the `"bounded"` method is quite similar to that of `LaplacesDemon`, with only two exceptions where `"direct"` proves to be superior. The cost of the input validation for the truncpois in relation to `LaplacesDemon` is a one-time cost which is more and more spread out as the number of samples requested, `n`, grows larger. Moreover, this cost is paid twice in the case of the `"inversion"` method, because the `rtruncpois()` function and the `qtruncpois()` function it calls within it do the input validation.

4.4 Experiment 4: numerical stability demonstration

A key advantage of `truncpois` is its ability to handle extreme parameter values without numerical underflow. This experiment demonstrates log-scale computations across a wide range of λ values, extended to include the stress-test extremes exercised by the package's own test suite: $\lambda = 100{,}000$ with a narrow truncation window, and the narrowest possible truncation window (b = a + 1).

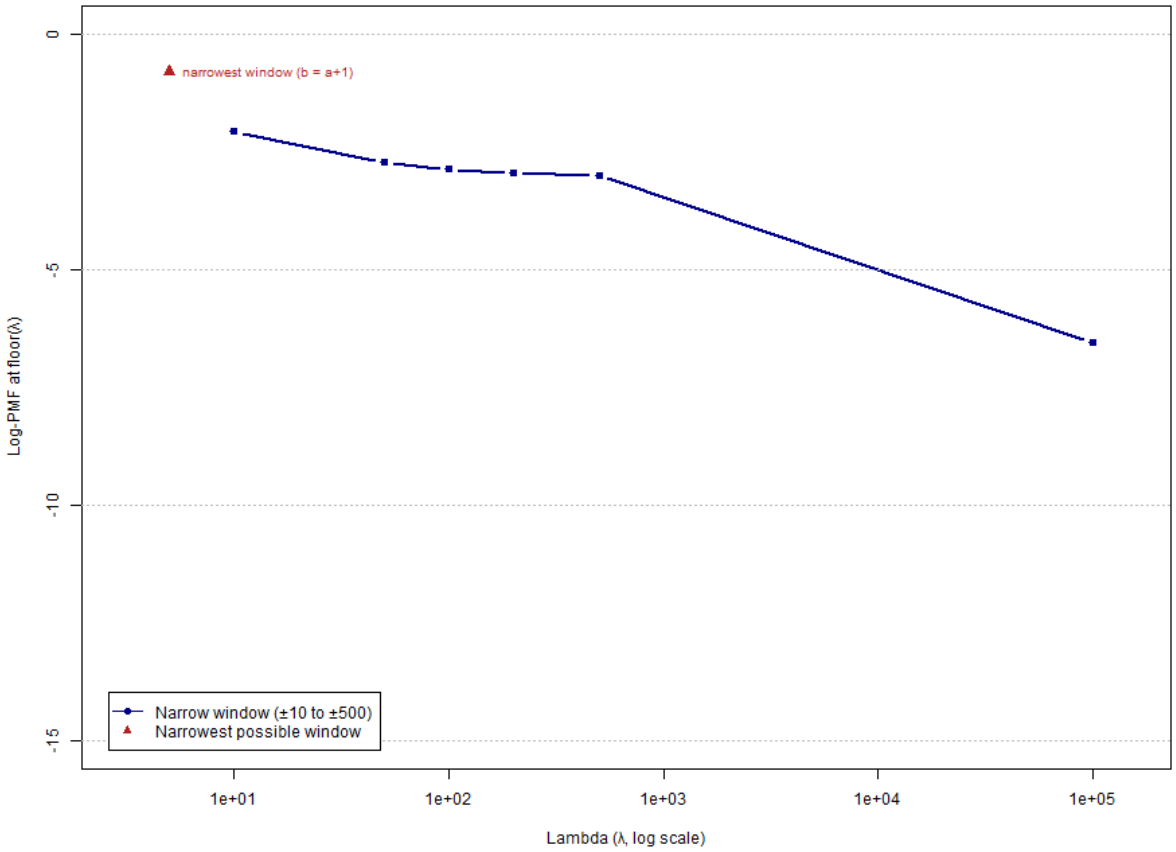

Figure 5: Numerical stability of `dtruncpois()` across extreme parameter values and truncation windows, including the narrowest possible window ($b = a + 1$).

The log-PMF remains computable for all tested λ values (10 to 100,000) with narrow truncation windows,

and the narrowest possible truncation window remains numerically well-behaved. Traditional naive implementations would produce zero or underflow errors in these regimes; `truncpois` handles them seamlessly via log-space arithmetic.

Table 3: Log-PMF at $\lfloor\lambda\rfloor$ for each tested parameter regime, computed via `dtruncpois(..., log = TRUE)`; for the narrowest-window row, $\lfloor\lambda\rfloor = 5$ falls outside $[a, b] = [3, 4]$, so the value shown is instead evaluated at $x = a$.

λ	a	b	Log-PMF at $\lfloor\lambda\rfloor$
10	0	20	-2.077
50	40	60	-2.730
100	90	110	-2.875
200	190	210	-2.956
500	490	510	-3.008
100,000	99,500	100,500	-6.555
100,000 (left-truncated only)	99,500	∞	-6.617
5 (narrowest window)	3	4	-0.811

These particular evaluation points sit close to the mode of each regime, and are thus not themselves at risk of underflow; the numerical-stability gain of computing entirely on the log scale instead becomes most apparent for evaluation of the normalizing constant $G(b) - G(a-1)$ in the case of windows placed further into the tails of very large-λ distributions, where a naive subtraction of two near-identical cumulative probabilities on the linear scale is prone to catastrophic cancellation, a risk that log-scale differencing (via the `.log_diff()` helper in `zzz_utils.R`) avoids entirely, irrespective of where the window is placed.

This experiment gives a chance to actually illustrate several defects in `LaplacesDemon` discussed in Section 2.2, rather than just asserting them, and to expand the numerical-stability comparison beyond `dtruncpois()` alone to `ptruncpois()` and `qtruncpois()` as well. Table 4 repeats the six parameter regimes above, adding another column for the log-PMF which `LaplacesDemon::dtrunc()` returns when a is passed directly, without the $a \to a-1$ correction a user unfamiliar with the discrete-truncation issue would naturally omit.

Table 4: Log-PMF at $\lfloor\lambda\rfloor$ from `truncpois` versus `LaplacesDemon` called with the uncorrected lower bound a.

λ	a	b	`truncpois` log-PMF	`LaplacesDemon` log-PMF (naive a)
10	0	20	-2.077	-2.077
50	40	60	-2.730	-2.704
100	90	110	-2.875	-2.839
200	190	210	-2.956	-2.914
500	490	510	-3.008	-2.962
100,000	99,500	100,500	-6.555	-6.555

The discrepancy is smallest in the first and last rows, where a is either 0 or so far into the tail that $G(a; \lambda)$ is already negligible, and largest in the middle rows, where a carries a non-trivial share of the

untruncated distribution's probability mass; in every such case, `LaplacesDemon` silently returns a subtly incorrect density rather than raising an error. A second, unrelated defect is demonstrated in Table 5: `LaplacesDemon::ptrunc()` accepts a `lower.tail` argument without error, but does not actually use it.

Table 5: `ptruncpois()` versus `LaplacesDemon::ptrunc()` for $P(X \leq 7 \mid 2 \leq X \leq 10, \lambda = 5)$, with `lower.tail = TRUE` and `lower.tail = FALSE`.

lower.tail	ptruncpois()	LaplacesDemon::ptrunc()
TRUE	0.873	0.873
FALSE	0.127	0.873

Passing `lower.tail = FALSE` simply returns the same value as the (lower-tail) default, confirming that the argument is simply ignored silently, not just undocumented; and of course this is a rather more dangerous failure mode than requiring a manual `1 - p` transformation, as there is no indication to the user that anything is wrong. The same defect affects `LaplacesDemon::qtrunc()`, and rather more seriously: Table 6 shows that it too ignores `lower.tail` silently (and unlike `ptrunc()` also does not accept `log.p` as an argument at all, raising the error `"p must be in [0,1]"` if a log-probability is supplied instead of either computing the correct log-scale quantile or failing with an informative message about the unsupported argument).

Table 6: `qtruncpois()` versus `LaplacesDemon::qtrunc()` for the 0.3 quantile of $X \mid 2 \leq X \leq 10, \lambda = 5$, with `lower.tail = TRUE` and `lower.tail = FALSE`.

lower.tail	qtruncpois()	LaplacesDemon::qtrunc()
TRUE	4	4
FALSE	6	4

4.5 Experiment 5: zero-truncated Poisson simulation

Zero-truncated Poisson (ZTP) distributions are widely used in count models where zero outcomes are structurally impossible to observe. For example, when hospitals record the number of visits made by individual patients, the resulting counts are zero-truncated, since a patient who never visited is not present in the hospital's records at all; similarly, the number of times an individual visits a tourist site is often recorded only on-site, so a zero-visit count is never observed either. This experiment simulates data from a ZTP distribution and demonstrates sampling, moment recovery, and model fit via visualisation.

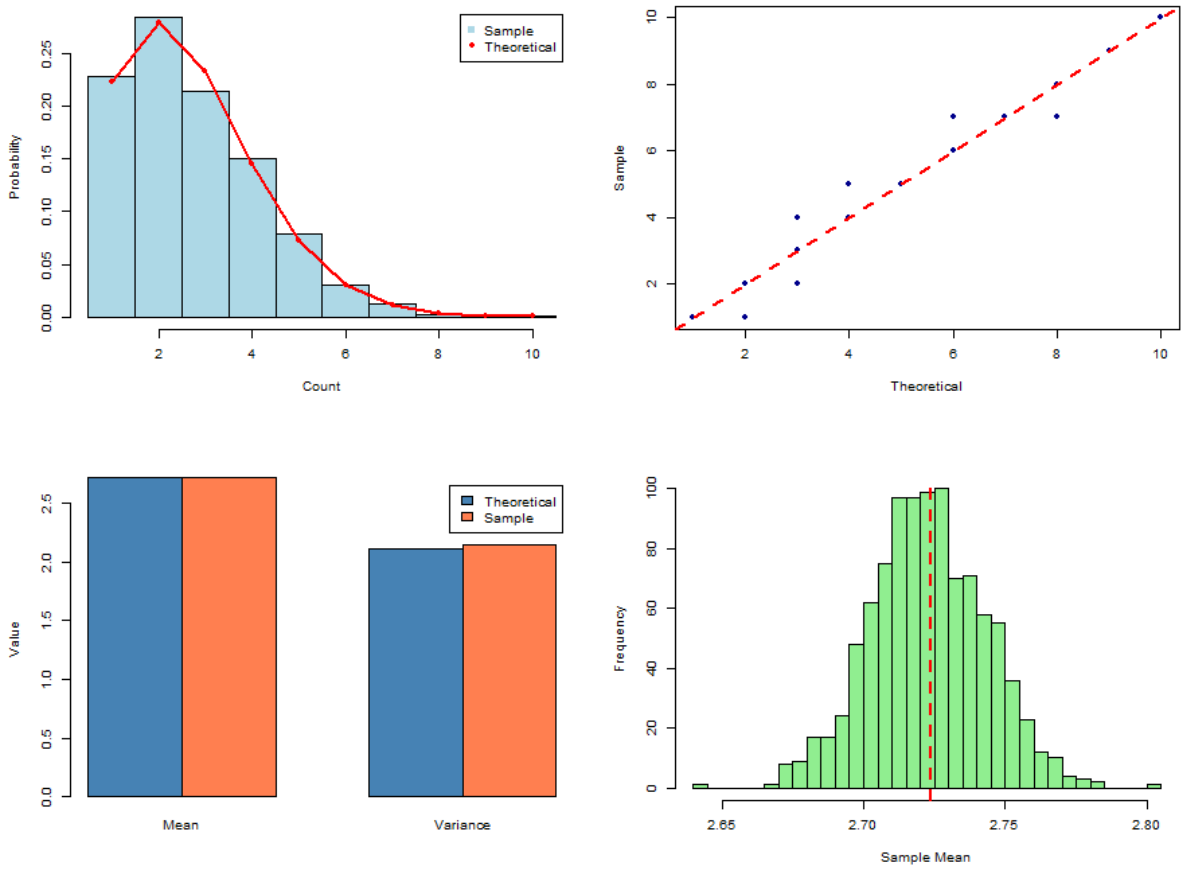

Figure 6: Zero-truncated Poisson simulation: sampling, moment recovery, and bootstrap inference.

The four panels show (clockwise from top-left): a histogram of 5,000 simulated ZTP samples with overlays of the theoretical PMF (top left); a Q-Q plot for comparing theoretical and simulated distributions (top right); a comparison of theoretical vs. empirical moments (bottom left); and a histogram of the bootstrap distribution of the sample mean (bottom right). All show good agreement between theory and simulation.

This scenario is representative of the applied use case motivating the package: a count variable (e.g. number of insurance claims, or number of hospital visits) that is only observed conditional on being at least 1. The standard, non-truncated Poisson model fails if fitted to such data without accounting for the missing zero category: it would systematically overestimate λ, since the observed sample mean of 2.72 (bootstrap distribution, right panel) is itself already a biased estimator of the *non-truncated* rate parameter; it instead estimates the conditional mean $E[X \mid X \geq 1]$, which is always strictly greater than λ itself. `extruncpois()` cannot be used to perform the correction: it computes the theoretical mean of the truncated distribution *given* a known λ, and is not itself an estimator of λ given the observed data. Recovering λ from an observed, zero-truncated sample instead requires `mletruncpois()` (demonstrated next in Experiment 6), because the sample mean of a zero-truncated sample is not, in general, equal to the maximum-likelihood estimate of λ.

4.6 Experiment 6: MLE parameter recovery

In this experiment, one can see how `mletruncpois()` function operates. This function is used to estimate the value of λ based on a sample of observed values. In the experiment, a sample is drawn from the truncated Poisson distribution with a known value of λ, and the value of λ is estimated using the sample by means of maximum likelihood.

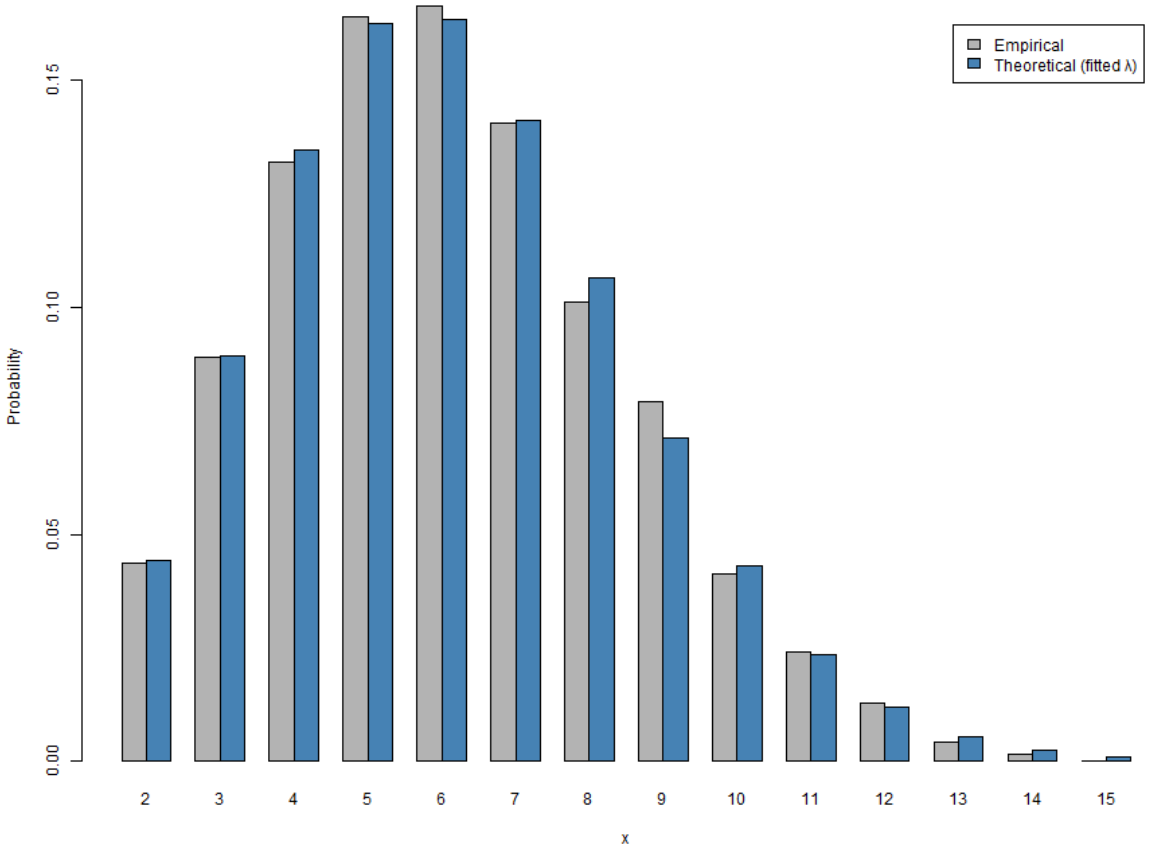

Figure 7: MLE parameter recovery: fitted versus true truncated Poisson distribution.

The maximum-likelihood estimate of λ recovered from the sample closely matches the true generating value, and the theoretical PMF evaluated at the fitted λ closely tracks the empirical sample proportions. This confirms that `rtruncpois()`, `mletruncpois()`, and the closed-form moment functions are mutually consistent.

The fitted value was $\hat{\lambda} = 6.037$ against a true generating $\lambda = 6$, from $n = 5{,}000$ samples (log-likelihood at $\hat{\lambda}$: $-11{,}184.6$). Table 7 compares the sample moments directly against the theoretical moments evaluated at $\hat{\lambda}$ via `extruncpois()`, `vartruncpois()`, `medtruncpois()`, and `modtruncpois()`.

Table 7: Sample moments of the simulated data versus theoretical moments evaluated at the fitted $\hat{\lambda} = 6.037$.

Statistic	Sample	Theoretical (at $\hat{\lambda}$)
Mean	6.120	6.120
Variance	5.497	5.615
Median	6	6
Mode	6	6

The close agreement across all four moments, particularly the mean, which matches to three decimal places since $\hat{\lambda}$ was itself chosen to maximize the likelihood of the observed sample, provides an internal consistency check spanning three separate parts of the package: random generation (`rtruncpois()`), estimation (`mletruncpois()`), and the closed-form moment functions, all evaluated independently of one another yet arriving at compatible answers.

5 Discussion

The `truncpois` R package provides a complete and numerically stable implementation of left-truncated, right-truncated, and doubly-truncated Poisson distributions. It allows for full distributional inference via the probability mass, cumulative distribution and quantile functions, as well as for random number generation, all with the naming convention of d/p/q/r that matches base `R` packages and all of which accept `log`, `log.p`, and `lower.tail` arguments. Closed-form expressions (from the Poisson recurrence relation) give the mean and variance exactly (i.e. without simulation error), while the median and mode (novel contributions of this package relative to Nadarajah and Kotz (2006)) are obtained using complementary exact, deterministic methods. All computation is carried out on the log scale, ensuring numerical stability in the presence of either extreme parameter values or narrow truncation windows. Three separate sampling algorithms are provided, each of which is best suited to a different combination of truncation width and sense (speed vs. memory) in which the computation is to be optimised. Maximum likelihood estimation of λ from count data (observations of the variable of interest) and its standard error are both provided by the `mletruncpois()` function. Through the use of a dedicated visualisation function, it is easy to compare the truncated and non-truncated distributions directly. Because all of these components are deterministic and exact as opposed to simulation based, all results are fully reproducible independent of any random seed except of course in cases where random sampling is itself the object of interest.

The experiments in Section 4 as a whole show that `truncpois` is both correct and stable across the parameter regimes it was designed for: PMF and CDF values match up with the usual distribution functions on default (untruncated) bounds, closed-form moments converge to their untruncated analogs at wide truncation windows, all three sampling algorithms converge to the correct closed-form moments at large enough sample sizes, log-scale computations behave nicely even at $\lambda = 100{,}000$ and with the truncate window as narrow as it can get, and the maximum-likelihood estimator recovers the true generating parameter from simulated data. Where there was a direct comparison to `LaplacesDemon`, it not only

confirms these advantages, but points out some concrete, previously unacknowledged defects in the alternative: an incorrect lower truncation boundary, and tail probability arguments that are silently ignored rather than just unsupported. Taken along with the literature review in Section 2.2, these results support the central claim of this thesis: that `truncpois` fills a real gap in R's ecosystem for exact, numerically stable, Poisson-specific truncated distribution computation.

There are a few limitations still worth stating explicitly. The package currently supports only the truncated Poisson distribution; many of same arguments about numerical stability and closed-form moments apply equally to other truncated discrete distributions (e.g., the negative binomial, binomial, or geometric), but those aren't yet implemented. A user with need for negative-binomial support, for instance, would have to look elsewhere (e.g., to the generic `truncdist` package), but caveats raised in Section 2.2 about `truncdist` and `LaplacesDemon` being inadequate for it apply equally to their use for those other truncated distributions, so this is not an exactly straightforward trade-up. Relatedly, `mletruncpois()` estimates a single scalar λ from a homogeneous sample with fixed, known truncation bounds; it does not natively support regression-style modelling where λ depends on covariates, unlike the dedicated regression frameworks `countreg::zerotrunc()` and `gamlss.tr`; to extend `truncpois` to a regression setting would require using quite different machinery (such as iteratively reweighted least squares or full likelihood optimisation over a coefficient vector), well beyond the one-dimensional `stats::optimize()` approach taken here. A further, harder extension would be to estimate the truncation bounds a and b themselves alongside λ, rather than assuming they are known in advance; this is a substantially more difficult problem that has only recently begun to receive attention in the statistics and theoretical computer science literature (Kontonis et al. 2019). The maximum-likelihood estimator itself is not in closed form, but rather based on optimisation; though the search interval is seeded near the sample mean and the truncated Poisson likelihood is well-behaved in practice (see Experiment 6), that has yet to be proven out formally to guarantee global convergence in every conceivable edge case, such as very small samples under heavy truncation.

All computation is done in base R, rather than via a compiled backend. This was considered outside the scope of this thesis, but a compiled implementation (e.g. via `Rcpp`) may offer some speed benefits for very large scale simulation workloads, particularly for the direct sampling method, whose cost scales with support width, as Experiment 3 shows. Finally, `truncpois` is currently only available via GitHub: it hasn't been submitted for `R CMD check --as-cran`, which unfortunately means it hasn't had the opportunity to expose the package to the kind of broader scrutiny (and edge case bug reports) that come from actually releasing on CRAN and being used publicly yet.

Several of the limitations noted above point to possible directions for future work. Extending the package's closed-form, log-scale approach to other truncated discrete distributions (in particular, the negative binomial and binomial) would generalise its core contribution beyond the particular case of the Poisson. Graphical diagnostics and goodness-of-fit tests, building on the existing `plottruncpois()` utility, would help practitioners assess whether a fit truncated Poisson model is a reasonable description of their own data. Finally, a compiled backend would extend the package's practical usefulness to simulation workloads

of substantially larger scale than currently possible in base `R` alone. In the meantime, `truncpois` is available at https://github.com/arunsundar022/truncpois, ready to be used by practitioners and to be extended by developers.

This work succeeds in providing an `R` implementation of the truncated Poisson distribution that is more complete, more numerically stable and more rigorously correct than any other, while being simple enough for a practitioner to use with the same d/p/q/r conventions and arguments as base `R`.

References

Brent, R. P. (1973), *Algorithms for Minimization Without Derivatives*, Englewood Cliffs, N.J.: Prentice-Hall.

Cameron, A. C., and Trivedi, P. K. (2001), "Essentials of count data regression," in *A companion to theoretical econometrics*, ed. B. H. Baltagi, Oxford: Blackwell, pp. 331–348.

Csárdi, G., Müller, K., and Hester, J. (2023), *Desc: Manipulate DESCRIPTION files*. https://doi.org/10.32614/CRAN.package.desc.

Gilbert, P., and Varadhan, R. (2019), *numDeriv: Accurate numerical derivatives*. https://doi.org/10.32614/CRAN.package.numDeriv.

Kontonis, V., Tzamos, C., and Zampetakis, M. (2019), "Efficient truncated statistics with unknown truncation," in *2019 IEEE 60th annual symposium on foundations of computer science (FOCS)*, IEEE, pp. 1578–1595.

Maechler, M. (2025), *Rmpfr: R MPFR - multiple precision floating-point reliable*. https://doi.org/10.32614/CRAN.package.Rmpfr.

Mersmann, O. (2024), *Microbenchmark: Accurate timing functions*. https://doi.org/10.32614/CRAN.package.microbenchmark.

Müller, K., Walthert, L., and Patil, I. (2025), *Styler: Non-invasive pretty printing of r code*. https://doi.org/10.32614/CRAN.package.styler.

Nadarajah, S., and Kotz, S. (2006), "R programs for computing truncated distributions," *Journal of Statistical Software*, 16, 1–8. https://doi.org/10.18637/jss.v016.c02.

Novomestky, F., and Nadarajah, S. (2016), *Truncdist: Truncated random variables*. https://doi.org/10.32614/CRAN.package.truncdist.

Padgham, M., Marks, K., de Bortoli, D., Csardi, G., Frick, H., Jones, O., Alexander, H., and Mowinckel, A. M. (2026), *Goodpractice: Advice on r package building*.

R Core Team (2026), *R: A language and environment for statistical computing*, Vienna, Austria: R Foundation for Statistical Computing.

Stasinopoulos, M., and Rigby, B. (2024), *Gamlss.tr: Generating and fitting truncated "gamlss.family" distributions*. https://doi.org/10.32614/CRAN.package.gamlss.tr.

Statisticat, LLC (2026), *LaplacesDemon: Complete environment for Bayesian inference*. https://doi.org/10.32614/CRAN.package.LaplacesDemon.

Wickham, H. (2011), "Testthat: Get started with testing," *The R Journal*, 3, 5–10.

Wickham, H., Bryan, J., Barrett, M., and Teucher, A. (2025), *Usethis: Automate package and project setup*. https://doi.org/10.32614/CRAN.package.usethis.

Wickham, H., Danenberg, P., Csárdi, G., and Eugster, M. (2026), *roxygen2: In-line documentation for r*. https://doi.org/10.32614/CRAN.package.roxygen2.

Wickham, H., Hester, J., Chang, W., and Bryan, J. (2022), *Devtools: Tools to make developing r packages easier*. https://doi.org/10.32614/CRAN.package.devtools.

Yee, T. (2025a), *VGAM: Vector generalized linear and additive models*. https://doi.org/10.32614/CRAN.package.VGAM.

Yee, T. (2025b), *VGAMdata: Data supporting the 'VGAM' package*. https://doi.org/10.32614/CRAN.package.VGAMdata.

Yellott, J. I., Jr. (1977), "The relationship between Luce's choice axiom, Thurstone's theory of comparative judgment, and the double exponential distribution," *Journal of Mathematical Psychology*, 15, 109–144.

Zeileis, A., and Kleiber, C. (2024), *Countreg: Count data regression*, R-Forge.

Zhang, X., and Reiter, J. P. (2022), "A generator for generalized inverse Gaussian distributions."

Appendix: Source Code

The complete source code, tests, documentation, and vignette for the truncpois R package are publicly available on GitHub for further reference.

Repository link: https://github.com/arunsundar022/truncpois